Bases de Datos 2

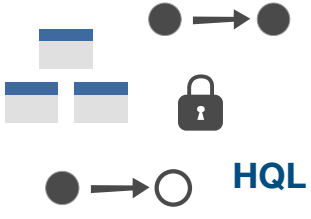
Teorías 03 - 05

Julián Grigera

Hibernate

Hibernate

Introducción



Hibernate es el ORM de referencia de JPA

Soporte para Jerarquías, Polimorfismo, Relaciones, Tx, Lazy, HQL

Hibernate
Mapeo XML

```
package prueba;
public class Persona {
      private Long idPersona;
      private String nombre;
      private Date fechaNacimiento;
      public Date getFechaNacimiento() {
         return fechaNacimiento;
      }
      public void
         setFechaNacimiento(Date
         fechaNacimiento) {
         this.fechaNacimiento =
         fechaNacimiento;
      }
      public Long getIdPersona() {
         return idPersona;
      }
      public void setIdPersona(Long
         idPersona) {
         this.idPersona = idPersona;
      }
      public String getNombre() {
         return nombre;
      }
      public void setNombre(String
         nombre) {
         this.nombre = nombre;
      }
}
```

Persona.hbm.xml (Archivo Mapping)

```
<hibernate-mapping package="prueba">
  <class name="Persona" table="PERSONA">

    <id name="idPersona" column="ID_PERSONA">
                <generator class="native"/>
    </id>


    <property name="nombre" not-null="true"/>


    <property name="fechaNacimiento"/>


  </class>
</hibernate-mapping>
```

Pueden definirse mapeos mediante XML
- Es más tedioso de definir y mantener que su alternativa (annotations)
+ Es mejor para la independencia (las clases del modelo quedan menos "contaminadas" por código de persistencia)

Hibernate
Mapeo Annotations

```
@Entity(name = "Contact")
public static class Contact {

        @Id
        private Integer id;

        private Name name;

        private String notes;

        private URL website;

        private boolean starred;

        //Getters and setters are omitted for brevity
}
```

Annotations permite un código más simple pero rompe el principio de independencia

---

Hibernate
POJOs

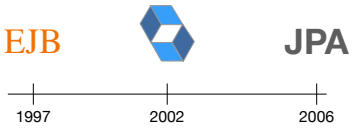
Reglas a seguir (POJO -> Plain Old Java Object)

- Implementar un **constructor sin argumentos**. *
- Proveer un **identificador**
- Clases **no-finales**
- **Setters y getters** para los campos persistentes

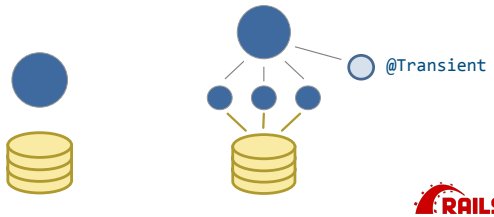
JPA
Java Persistence API

JPA es un estándar Basado en Hibernate, EJB y otros productos.
Hibernate es la implementación de referencia, si bien existen otras.

Java Persistence API
JPA



Entities
Mapeo con JPA



RAILS
CONVENTION OVER CONFIGURATION

Entidades en JPA: POJOs persistentes.
Por default en JPA se persisten todos los atributos, a menos que se marquen como @Transient - un ejemplo de Convention over Configuration

JPA - Mapeos

Mapeo con JPA

```
@Entity  6 usages
@Table(name = "reservas")
public class Reserva {
    @{...}
    private long id;
```

```
protected Date fecha;  3 usages

@Column(name = "desc")  2 usages
protected String descripcion;

@Transient  no usages
protected int diasRestantes;
```

Ejemplos de código

- Las entidades tienen opciones para configurar, como el nombre de la tabla (útil para evitar palabras reservadas del DBMS)
- Los atributos también, pueden especificarse propiedades de la columna, restricciones, etc
- También se puede ligar @Column a un getter/método

Jerarquías

Mapeo con JPA

SINGLE_TABLE

```
@Entity(name = "clientes")  7 usages
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo")
public abstract class  Cliente {
    // ...
}
```

```
@Entity  3 usages
@DiscriminatorValue("1")
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  5 usages
@DiscriminatorValue("2")
public class ClienteVIP extends Cliente {

    // ...
}
```

Jerarquías

Mapeo con JPA

JOINED

```
@Entity(name = "clientes")  7 usages
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class  Cliente {

    // ...
}
```

```
@Entity  3 usages
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  3 usages
public class ClienteVIP extends Cliente{

    // ...
}
```

Jerarquías
Mapeo con JPA

TABLE_PER_CLASS

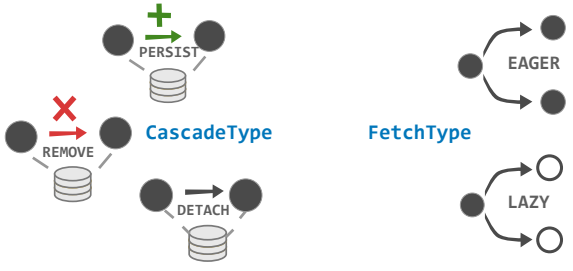
```
@Entity  8 usages
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class  Cliente {
```

```
@Entity  3 usages
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  3 usages
public class ClienteVIP extends Cliente{

    // ...
}
```

En TABLE_PER_CLASS la superclase se marca como Entity pero no se indica nada relacionado a la tabla - dado que no se generará

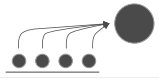
Relaciones
Mapeo con JPA



- **Cascade** establece la persistencia por alcance. Por defecto nada se propaga en JPA. Puede ser ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH
- El tipo de **Fetch** puede ser **EAGER** o **LAZY**, cada uno puede ser más o menos performante según el caso.
- El valor de tanto Cascade como Fetch dependerá del uso.

Relaciones
Many to Many - to One

```
@Entity
@Table(name = "BOOKS")
public class Book {

    @ManyToMany
    private Set<Author> authors;

    ...
}
```



```
@Entity
public class Book {

    @ManyToOne(fetch = FetchType.LAZY)
    private Publisher publisher;

    ...
}
```

ManyToMany: No requiere especificación de tipo (como Hibernate)
ManyToOne

Relaciones

One to Many

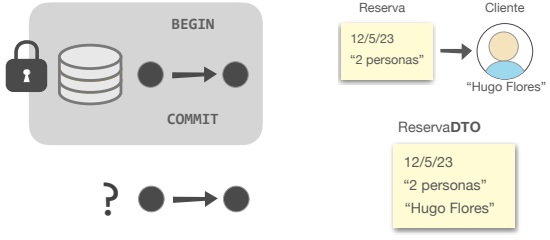
```java
@Entity(name = "Post")
@Table(name = "post")
public class Post {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @OneToMany(
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<PostComment> comments = new ArrayList<>();
```

```java
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private List<PostComment> comments = new ArrayList<>();
```

Mapeo por default (tablaintermedia)
Join Column

DTOs

JPA



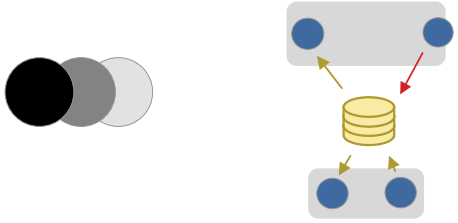
- No puedo leer objetos fuera de una tx, qué hago?
- DTOs. Según Fowler: "An object that carries data between processes in order to reduce the number of method calls."
- Se puede aprovechar el hecho de crear otra clase y cambiar el formato, agregar atributos externos, etc.

Migrations

Actualización de Bases de Datos en ORMs

Al usar ORMs tenemos que considerar la evolución
Existe el concepto de migraciones - pensarlo como el historial "commits" en la estructura de la BD

Concurrencia
JPA



¿Cómo se manejan los problemas de concurrencia?
Se observa el @Version al escribir,
- si el nro es el mismo de la base, ok
- si no, tengo una versión vieja << hasta ahí llega JPA - solo eleva excepción

---

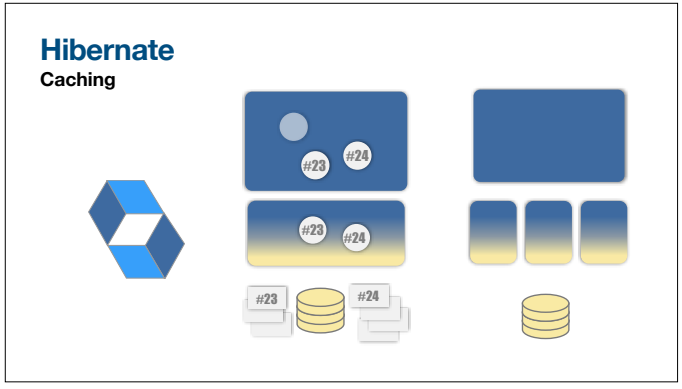
Hibernate
Proxies y Cachés

JPA es un estándar Basado en Hibernate, EJB y otros productos.
Hibernate es la implementación de referencia, si bien existen otras.

Hibernate
Proxies

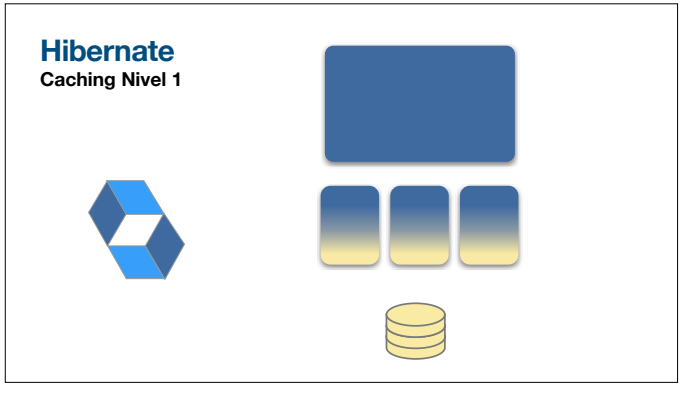
Qué pasa si…
1. Abro una sesión, una tx y guardo un objeto “reserva” relacionado a otro, “cliente” de forma LAZY
2. Abro otra tx y recupera “reserva”
3. Pido el “cliente” a la reserva recuperada
4. Cierro sesión

El cliente, debería ser un proxy? O un objeto del modelo?
Y si hago los pasos 2-3 en una nueva sesión?



Tiene soporte para:

- Caching
    - Nivel 1 - activado por default, depende de cada sesión
    - Nivel 2 - solamente un hotspot, con implementaciones para descargar
        - Incluye caching de queries

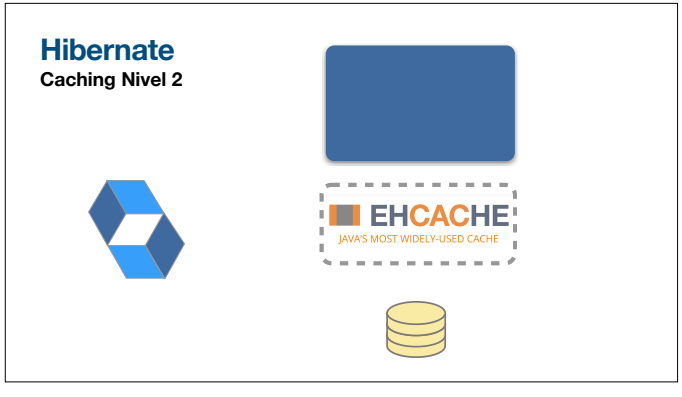


Nivel 1

Cuando una entidad se carga o actualiza por primera vez en una sesión, se almacena en la caché a nivel de sesión.

Solicitudes posteriores de la misma entidad en la misma sesión se sirven desde la caché

Se minimizan así accesos a la BD



Nivel 2

A nivel de SessionFactory - si no se halla un objeto en la **CacheL1**, se busca en **CacheL2**

No viene implementada, pero hay productos que lo hacen

Se pueden cachear resultados de consultas HQL

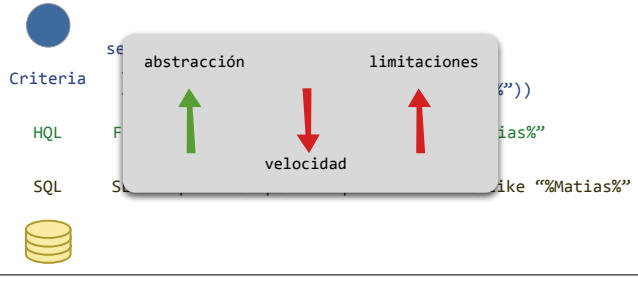
Consultas

Lenguaje de Consulta
OQL

SQL	OQL
Select <atributos>	Select mensajes
From <tabla,tablas>	From Clase
Where <condiciones>	Where mensajes

Ca. 2000 OMG (Object Management Group) publica un estándar para el acceso a las BBDD OO: **OQL (Object Query Language)**
Tomaron como base **SQL** - buscando mejorar la adopción
Se basa en la “extensión” de clase (Extent)

Lenguajes de Consulta
En Hibernate

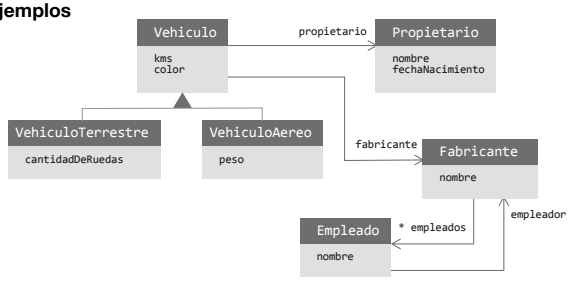


No es fácil implementar OQL, por lo que Hibernate ofrece:

- **SQL** es rápido, pero es un string (debugging complejo) y está atado a la BD 100% (tablas en vez de clases mapeadas). Puede formar objetos con la respuesta.
- **HQL** clases en el FROM. No puedo usar mensajes, solo atributos. Sigue siendo un string.
- **Criteria** se construye la query en objetos. Es más *debugger-friendly*. Más lento.

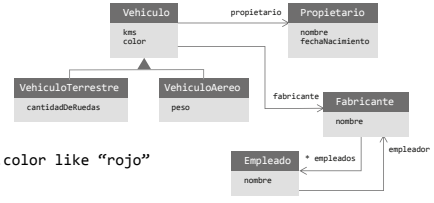
HQL

Ejemplos



HQL

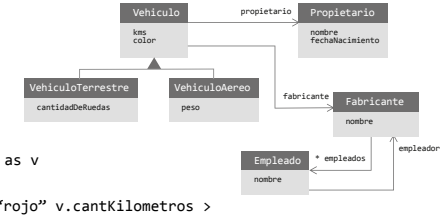
Ejemplos




```
from Vehiculo

from Vehículo v where v.color like “rojo”

select vehiculo
   from Vehiculo vehiculo join vehiculo.fabricante fabricante
   where fabricante.nombre like “Honda”

select v
   from Vehiculo v join v.propietario p join v.fabricante f
   where v.color like "rojo" and v.cantKilometros > 20000
         and f.nombre like "honda" and p.nombre like "juan"
```

HQL

Ejemplos




```
select v

from VehiculoTerrestre as v

   where v.color like “rojo” v.cantKilometros >

      (select avg(v1.cantKilometros) from Vehiculo as v1 )
```

HQL
Tipos de Retorno

```
select reserva, cliente
   from Reserva reserva where ...
```

```
select new Cliente(nombre, fecha)
   from Cliente c
   where ...
```

También se pueden retornar arrays de listas y más

HQL
Funciones de Agregación

```
avg(...), sum(...), min(...), max(...)
count(*)
count(...), count(distinct ...), count(all...)
```

HQL
Subconsultas

```
from Cat as fatcat
where fatcat.weight > (select avg(cat.weight) from DomesticCat cat)
```

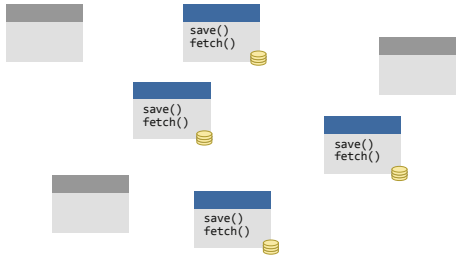
```
from Cat as cat
where not exists ( from Cat as mate where mate.mate = cat)
```

```
from DomesticCat as cat
where cat.name not in
   (select name.nickName from Name as name)
```

Patrones de Persistencia

Persistencia pre-ORM

¿Cómo se modelaría?

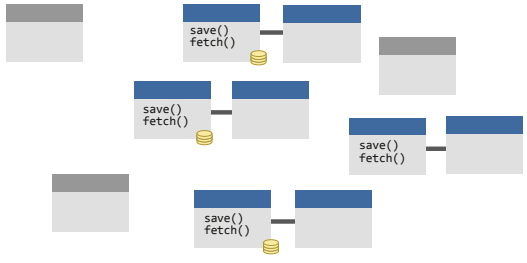


¿Cómo persistir un modelo de OO si no conociéramos los ORM?
Caso naïve - cada clase se ocupa de su “guardar”, “obtener”, etc
Problemas:

1. Necesito una instancia de cada una para que funcione
2. Acoplamiento con la tecnología de persistencia

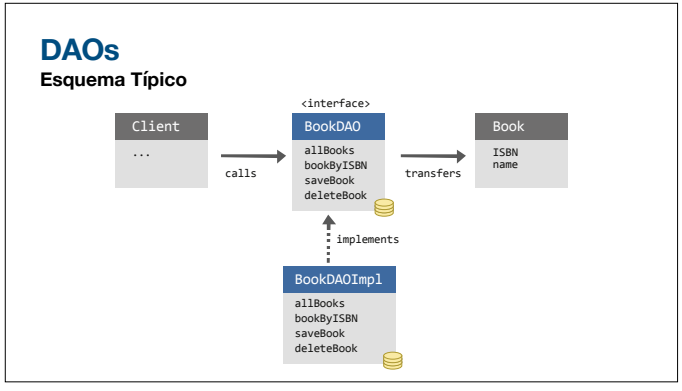
DAOs

Data Access Object

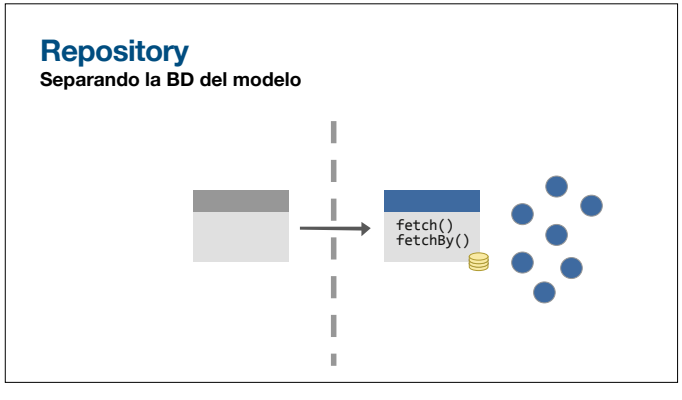


DAOs: establece que por cada objeto de dominio persistente debe existir una clase que persiste y recupera sus instancias.
Problemas:

1. Granularidad / relaciones entre DAOs de clases relacionadas
2. “Mudanza” del comportamiento de negocio a la capa de persistencia / el modelo queda anémico
3. Acoplamiento



- Aparecen los ORMs ¿Dónde iría el ORM en este esquema?
- Se solía poner en el DAO, cosa que desaprovecha las capacidades de todo ORM.



El patrón Repository ya no representa la **capa de persistencia**, sino la **colección** de objetos persistentes.
Se enfoca en el “fetch” - aprovecha los updates de ORM
Puede emular las colecciones de Smalltalk y sus filtros y usos de bloques (ej. findByEmail, getUsersCount)
Puede complicarse con el tipado estático - la implementación de puede volver repetitiva

---